Micron AI Agentic Solutions Interview Preparation Guide

Khushal Bhalia

Comprehensive Study Material

March 16, 2026

Covering: Transformers, LLMs, AI Agents, RAG, Multi-Agent Systems, Agentic Frameworks, Vector Databases, and Project Deep-Dives

Contents

1 Word2Vec and Word Embeddings **2**
- 1.1 What Is Word2Vec? . . . 2
- 1.2 Two Architectures . . . 2
- 1.3 Training Details . . . 2
- 1.4 Limitations of Word2Vec . . . 2

2 Tokenization and Embeddings **2**
- 2.1 Why Tokenization Matters . . . 3
- 2.2 Tokenization Methods . . . 3
- 2.3 Embeddings in Transformers . . . 3

3 Transformer Architecture (In Depth) **3**
- 3.1 High-Level Overview . . . 3
- 3.2 Encoder Block (Detailed) . . . 4
- 3.3 Decoder Block (Detailed) . . . 4
- 3.4 Self-Attention Mechanism (Core of Transformers) . . . 4
  - 3.4.1 Why Q, K, V? . . . 5
- 3.5 Multi-Head Attention . . . 5
- 3.6 Feed-Forward Network (FFN) . . . 5
- 3.7 Residual Connections and Layer Normalization . . . 5
- 3.8 Masked Self-Attention in Decoders . . . 5
- 3.9 Encoder-Decoder vs. Decoder-Only vs. Encoder-Only . . . 6

4 LLMs vs. SLMs (Large vs. Small Language Models) **6**
- 4.1 Definitions . . . 6
- 4.2 Comparison . . . 6
- 4.3 When to Use Which . . . 6

5 Training Paradigms **6**
- 5.1 Pre-training . . . 7
- 5.2 Fine-tuning . . . 7
- 5.3 Alignment: RLHF and DPO . . . 7
- 5.4 Inference-Time Optimization . . . 7

6 Scaling Laws **7**
- 6.1 Kaplan / Chinchilla Scaling Laws . . . 7
- 6.2 Key Insights . . . 8

7 Multilingualism and Multimodality **8**
- 7.1 Multilingual Models . . . 8
- 7.2 Multimodal Models . . . 8

8 Mixture of Experts (MoE) **8**
- 8.1 Concept . . . 8
- 8.2 Router/Gating Network . . . 8
- 8.3 Examples and Key Points . . . 9

9 Vector Databases **9**
9.1 What Are They? . . . 9
9.2 Key Concepts . . . 9
9.3 Indexing Algorithms . . . 9
9.4 Popular Vector Databases . . . 9

10 RAG (Retrieval-Augmented Generation) **9**
10.1 The Problem RAG Solves . . . 9
10.2 RAG Pipeline . . . 10
10.3 Key Components . . . 10
10.4 Advanced RAG Techniques . . . 10

11 Indexing and Searching (General) **10**
11.1 Traditional Indexing . . . 10
11.2 Text Search . . . 10
11.3 Semantic Search . . . 10
11.4 Hybrid Search . . . 11

12 Tool Calling **11**
12.1 What Is Tool Calling? . . . 11
12.2 How It Works . . . 11
12.3 Example (OpenAI Function Calling) . . . 11

13 A2A (Agent-to-Agent Protocol) **11**
13.1 What Is A2A? . . . 11
13.2 Key Concepts . . . 11
13.3 A2A vs. MCP . . . 12

14 MCP (Model Context Protocol) **12**
14.1 What Is MCP? . . . 12
14.2 Architecture . . . 12
14.3 What MCP Servers Expose . . . 12
14.4 Transport Protocols . . . 12
14.5 Why MCP Matters . . . 12

15 Agentic AI Frameworks **12**
15.1 LangChain . . . 12
15.2 LangGraph . . . 13
15.3 CrewAI . . . 13
15.4 AutoGen (Microsoft) . . . 13
15.5 n8n . . . 13
15.6 Framework Comparison . . . 14

16 Orchestration **14**
16.1 What Is Agent Orchestration? . . . 14
16.2 Orchestration Patterns . . . 14
16.3 Key Orchestration Concerns . . . 14

17 Memory in AI Agents **14**
17.1 Types of Memory . . . 14
17.2 Memory in LangChain . . . 14

18 Multi-Agent Systems **15**
18.1 Why Multi-Agent? . . . 15
18.2 Common Multi-Agent Patterns . . . 15
18.3 Design Considerations . . . 15

19 Project 1: ClassQR — AI-Powered Attendance System **16**
19.1 System Overview . . . 16
19.2 Database Schema . . . 16
19.3 LangChain + PostgreSQL Integration (Node.js) . . . 17

20 Project 2: Motive Video Search — Hybrid Semantic Search **20**
20.1 Problem Statement . . . 20
20.2 Approaches Tried . . . 20
20.3 Hybrid Search Code (Python) . . . 20

21 Potential Interview Questions and Answers **23**
21.1 Transformer / LLM Architecture . . . 23
21.2 RAG and Agents . . . 23
21.3 Agent-Specific Questions . . . 24

22 Quick Reference: Key Formulas and Concepts **26**
22.1 Attention . . . 26
22.2 Multi-Head Attention . . . 26
22.3 Positional Encoding . . . 26
22.4 Cosine Similarity . . . 26
22.5 TF-IDF . . . 26
22.6 Chinchilla Scaling . . . 26
22.7 MoE Gating . . . 26
22.8 LoRA . . . 26

1 Word2Vec and Word Embeddings

1.1 What Is Word2Vec?

Word2Vec (Mikolov et al., 2013, Google) learns dense vector representations of words from large corpora. Unlike one-hot encoding, these vectors capture **semantic relationships**: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$.

1.2 Two Architectures

CBOW (Continuous Bag of Words)

- **Input:** Context words (surrounding words).
- **Output:** Predict the center/target word.
- Faster to train; works well on frequent words.
- Example: "The cat <u>sat</u> on the mat" — given {The, cat, on, the}, predict "sat".

Skip-gram

- **Input:** The center/target word.
- **Output:** Predict the context words.
- Slower but better for rare words and small corpora.
- Example: Given "sat", predict {The, cat, on, the, mat}.

1.3 Training Details

- **Window Size:** How many surrounding words to consider (typically 2–10).
- **Negative Sampling:** Instead of updating all weights, sample a few "negative" (random) words. This makes training tractable for large vocabularies.
- **Hierarchical Softmax:** Uses a binary tree to reduce softmax computation from $O(V)$ to $O(\log V)$.
- Typical dimensionality: 100–300.

1.4 Limitations of Word2Vec

- **Static embeddings**: One vector per word regardless of context ("bank" in "river bank" vs. "bank account" get the same vector).
- No sub-word information (misspellings and morphology are not handled).
- Solved by: **FastText** (sub-word n-grams), **ELMo** (contextual from BiLSTM), **BERT** (contextual from Transformers).

Interview Tip

Be ready to explain the difference between static embeddings (Word2Vec, GloVe) and contextual embeddings (ELMo, BERT). Also know: GloVe uses matrix factorization on co-occurrence matrix; Word2Vec uses a shallow neural network.

2 Tokenization and Embeddings

2.1 Why Tokenization Matters

Models cannot process raw text; they need numerical tokens. The tokenizer converts text → token IDs → embeddings.

2.2 Tokenization Methods

Tokenization Approaches

1. **Word-level:** Split by whitespace/punctuation. Simple but huge vocabulary; OOV (out-of-vocabulary) problem.
2. **Character-level:** Each character is a token. Small vocab but very long sequences; loses word-level semantics.
3. **Sub-word (BPE – Byte Pair Encoding):** Used by GPT. Start with character vocabulary; iteratively merge the most frequent adjacent pairs. Balances vocabulary size and sequence length.
4. **WordPiece:** Used by BERT. Similar to BPE but merges based on likelihood increase rather than frequency.
5. **Unigram / SentencePiece:** Used by T5, LLaMA. Starts with a large vocabulary and iteratively prunes tokens that contribute least to likelihood.

2.3 Embeddings in Transformers

In a Transformer, each token ID maps to a learned embedding vector via a lookup table (embedding matrix $E \in \mathbb{R}^{V \times d}$):

$$\text{token_id} \xrightarrow{\text{lookup}} \mathbf{e}_i \in \mathbb{R}^d$$

Additionally, **positional encodings** are added so the model knows token order:

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i$$

Sinusoidal Positional Encoding (original Transformer):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{1}$$

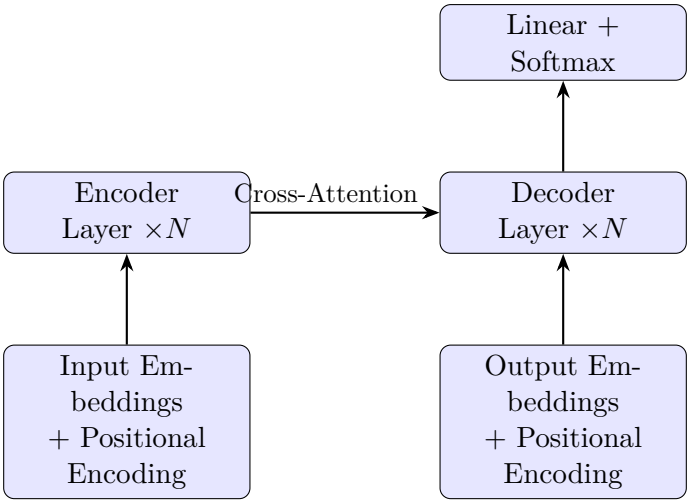
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{2}$$

Modern models (GPT, LLaMA) use **Rotary Position Embeddings (RoPE)** which encode relative positions via rotation matrices applied in the attention computation.

3 Transformer Architecture (In Depth)

3.1 High-Level Overview

The Transformer (Vaswani et al., 2017 — "Attention Is All You Need") replaced recurrence with **self-attention**, enabling parallelism and capturing long-range dependencies.



3.2 Encoder Block (Detailed)

Each encoder layer consists of:

1. **Multi-Head Self-Attention (MHSA)**
2. **Add & Norm** (Residual connection + Layer Normalization)
3. **Feed-Forward Network (FFN)** — two linear layers with ReLU/GELU
4. **Add & Norm**

3.3 Decoder Block (Detailed)

Each decoder layer has:

1. **Masked Multi-Head Self-Attention** (prevents attending to future tokens)
2. Add & Norm
3. **Multi-Head Cross-Attention** (attends to encoder output)
4. Add & Norm
5. Feed-Forward Network
6. Add & Norm

3.4 Self-Attention Mechanism (Core of Transformers)

Scaled Dot-Product Attention

Given an input sequence $X \in \mathbb{R}^{n \times d}$:

Step 1: Project into Q, K, V

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

where $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ and $W^V \in \mathbb{R}^{d \times d_v}$.

Step 2: Compute Attention Scores

$$\text{score}(Q, K) = \frac{QK^\top}{\sqrt{d_k}}$$

The $\sqrt{d_k}$ scaling prevents dot products from growing too large (which would push softmax into saturation regions with vanishing gradients).

Step 3: Apply Softmax

$$\alpha = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)$$

Each row of α is an attention distribution over all positions.
Step 4: Weighted Sum of Values

$$\text{Attention}(Q, K, V) = \alpha \cdot V = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

3.4.1 Why Q, K, V?

- **Query (Q):** "What am I looking for?" — represents the current token's information need.
- **Key (K):** "What do I contain?" — represents what each token offers.
- **Value (V):** "What information do I actually provide?" — the content to aggregate.
- Analogy: Like a database lookup — Query matches against Keys to find relevant Values.

3.5 Multi-Head Attention

Instead of one attention function, use h parallel heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Why multiple heads? Each head can learn different types of relationships:

- Head 1 might learn syntactic dependencies (subject-verb).
- Head 2 might learn coreference (pronoun to noun).
- Head 3 might learn positional patterns.

Typical: $d_{\text{model}} = 512$, $h = 8$ heads, so $d_k = d_v = 512/8 = 64$.

3.6 Feed-Forward Network (FFN)

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Usually $d_{\text{ff}} = 4 \times d_{\text{model}}$ (e.g., 2048 for $d_{\text{model}} = 512$).

The FFN processes each position independently — it is where the model stores "knowledge" (factual information from training).

3.7 Residual Connections and Layer Normalization

$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

- **Residual connections:** Allow gradients to flow directly; prevent vanishing gradients in deep networks.
- **Layer Normalization:** Normalizes across features (not batch); stabilizes training.
- Modern LLMs use **Pre-Norm** (normalize before sublayer) instead of Post-Norm for more stable training.
- Many use **RMSNorm** (simpler, no mean subtraction) instead of LayerNorm.

3.8 Masked Self-Attention in Decoders

In autoregressive generation, the model must not see future tokens:

$$\text{mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Applied before softmax: $\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{mask}\right)$.

3.9 Encoder-Decoder vs. Decoder-Only vs. Encoder-Only

Architecture	Example Models	Best For	Attention Type
Encoder-only	BERT, RoBERTa	Classification, NER	Bidirectional
Decoder-only	GPT, LLaMA, Mistral	Text generation	Causal (left-to-right)
Encoder-Decoder	T5, BART, mBART	Translation, Summarization	Full + Causal

Interview Tip — Attention Deep Dive

Be prepared to:

- Write the attention formula from scratch.
- Explain why we scale by $\sqrt{d_k}$.
- Explain the difference between self-attention and cross-attention.
- Explain the masking mechanism in decoders.
- Discuss attention complexity $O(n^2 d)$ and efficient alternatives (Flash Attention, Sparse Attention, Sliding Window Attention used in Mistral).

4 LLMs vs. SLMs (Large vs. Small Language Models)

4.1 Definitions

- **LLM (Large Language Model):** 7B+ parameters. Examples: GPT-4 (∼1.8T), LLaMA-3 70B, Claude 3.5.
- **SLM (Small Language Model):** $<$ 7B parameters. Examples: Phi-3 (3.8B), Gemma 2B, Mistral 7B, TinyLlama 1.1B.

4.2 Comparison

Aspect	LLMs	SLMs
Parameters	7B – 1.8T	1B – 7B
Training Data	Trillions of tokens	Curated, smaller datasets
Inference Cost	High (needs GPUs)	Low (can run on edge)
Latency	Higher	Lower
Accuracy (general)	Higher	Task-dependent
Fine-tuning Cost	Expensive	Affordable
Deployment	Cloud/API	On-device, edge, local
Use Case	General-purpose	Domain-specific, embedded

4.3 When to Use Which

- **Use LLMs:** Complex reasoning, multi-step tasks, code generation, general assistants.
- **Use SLMs:** Edge deployment (phones, IoT), low-latency needs, privacy-sensitive (on-prem), cost-sensitive applications, domain-specific fine-tuned tasks.
- **Micron relevance:** As a memory company, Micron is interested in running AI models efficiently on-device, making SLMs very relevant.

5 Training Paradigms

5.1 Pre-training

- Train on massive unlabeled corpora (internet text).
- **Causal Language Modeling (CLM):** Predict next token. Used by GPT family.
- **Masked Language Modeling (MLM):** Mask 15% of tokens, predict them. Used by BERT.
- **Span Corruption:** Mask spans of tokens, generate them. Used by T5.
- Cost: Millions of dollars, weeks on thousands of GPUs.

5.2 Fine-tuning

- Adapt a pre-trained model to a specific task or domain.
- **Full fine-tuning:** Update all parameters. Best results but expensive.
- **Parameter-Efficient Fine-Tuning (PEFT):**
  - **LoRA (Low-Rank Adaptation):** Add low-rank matrices $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times d}$, $r \ll d$. Only train A and B.
  - **QLoRA:** LoRA + 4-bit quantized base model. Enables fine-tuning 65B models on a single 48GB GPU.
  - **Adapters:** Small bottleneck layers inserted into Transformer blocks.
  - **Prefix Tuning:** Learn "virtual prefix" tokens prepended to keys/values.

5.3 Alignment: RLHF and DPO

- **Supervised Fine-Tuning (SFT):** Train on human-written instruction-response pairs.
- **RLHF (Reinforcement Learning from Human Feedback):**
  1. Train a reward model on human preference data (response A vs. B).
  2. Optimize the LLM using PPO (Proximal Policy Optimization) to maximize the reward.
- **DPO (Direct Preference Optimization):** Skip the reward model; directly optimize the LLM on preference pairs using a classification-like loss. Simpler and increasingly popular.

5.4 Inference-Time Optimization

- **Quantization:** Reduce precision (FP32 $\rightarrow$ INT8/INT4). GPTQ, AWQ, GGUF formats.
- **KV-Cache:** Cache key/value matrices across generation steps to avoid recomputation.
- **Speculative Decoding:** Use a small draft model to generate candidate tokens; large model verifies in parallel.
- **Flash Attention:** Tiling-based attention that reduces memory from $O(n^2)$ to $O(n)$ by avoiding materializing the full attention matrix.

6 Scaling Laws

6.1 Kaplan / Chinchilla Scaling Laws

Chinchilla Scaling Law (Hoffmann et al., 2022)

For compute-optimal training:

$$D \approx 20 \times N$$

where N = number of parameters, D = number of training tokens.
A 10B parameter model should be trained on ~200B tokens.
The original GPT-3 (175B params, 300B tokens) was **undertrained** by this law. Chin-

chilla (70B params, 1.4T tokens) matched GPT-3 performance with less compute.

6.2 Key Insights

- Loss follows a **power law** with compute, data, and model size:

$$L(N, D) \propto N^{-\alpha} + D^{-\beta} + \text{irreducible loss}$$

- There are diminishing returns; doubling parameters doesn't halve the loss.
- **Emergent abilities**: Some capabilities (chain-of-thought, in-context learning) appear only above certain model sizes.
- Modern trend: Over-train smaller models (LLaMA-3 8B trained on 15T tokens, far exceeding Chinchilla optimal) for better inference efficiency.

7 Multilingualism and Multimodality

7.1 Multilingual Models

- **mBERT, XLM-R:** Encoder models trained on 100+ languages.
- **mBART, mT5:** Encoder-decoder for multilingual generation/translation.
- **Cross-lingual transfer:** Train on English, evaluate on Hindi — works because of shared sub-word tokens and aligned representations.
- **Challenges:** Curse of multilinguality (capacity dilution), low-resource languages under-represented, tokenizer bias (non-Latin scripts get more tokens per word).

7.2 Multimodal Models

- **Vision-Language:** GPT-4V, LLaVA, Gemini — process both images and text.
- **Architecture:** Vision encoder (ViT/CLIP) produces image tokens; these are projected into the LLM's embedding space and concatenated with text tokens.
- **CLIP (Contrastive Language–Image Pre-training):** Learns aligned image-text embeddings via contrastive loss. Used for zero-shot image classification and retrieval.
- **Audio:** Whisper (speech-to-text), AudioLM.
- **Video:** Typically frame sampling + vision encoder per frame.

8 Mixture of Experts (MoE)

8.1 Concept

Instead of one large FFN, have E expert FFNs but only activate k per token (sparse activation):

$$y = \sum_{i=1}^{E} g_i(x) \cdot \text{Expert}_i(x), \quad \text{where only top-}k\ g_i \neq 0$$

8.2 Router/Gating Network

$$g(x) = \text{TopK}(\text{softmax}(W_g \cdot x))$$

The router decides which experts to activate for each token.

8.3 Examples and Key Points

- **Mixtral 8x7B:** 8 experts, top-2 routing. Total params: 46.7B, active per token: ∼12.9B. Performance comparable to LLaMA-2 70B.
- **GPT-4:** Rumored to use MoE with 8 experts of ∼220B each.
- **Benefits:** More parameters (knowledge capacity) without proportional compute increase.
- **Challenges:** Load balancing (some experts get all tokens), training instability, higher memory (all experts must be in memory even if not all are active).
- **Auxiliary loss:** Added during training to encourage balanced expert utilization.

9 Vector Databases

9.1 What Are They?

Databases optimized for storing, indexing, and querying high-dimensional vectors (embeddings). Essential for similarity search in AI applications.

9.2 Key Concepts

- **Embedding:** Dense vector representation of text/image/audio (e.g., 768-d or 1536-d).
- **Similarity Metrics:**
  - **Cosine Similarity:** $\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$ — measures angle; ignores magnitude.
  - **Euclidean Distance (L2):** $\|\mathbf{a} - \mathbf{b}\|_2$ — measures absolute distance.
  - **Dot Product:** $\mathbf{a} \cdot \mathbf{b}$ — used when magnitude matters.
- **Approximate Nearest Neighbor (ANN):** Exact search is $O(n)$; ANN algorithms trade accuracy for speed.

9.3 Indexing Algorithms

- **HNSW (Hierarchical Navigable Small World):** Graph-based. Fast queries, high recall. Used by most vector DBs.
- **IVF (Inverted File Index):** Cluster vectors; search only relevant clusters.
- **PQ (Product Quantization):** Compress vectors for memory efficiency.
- **FAISS:** Facebook's library combining IVF + PQ + HNSW.

9.4 Popular Vector Databases

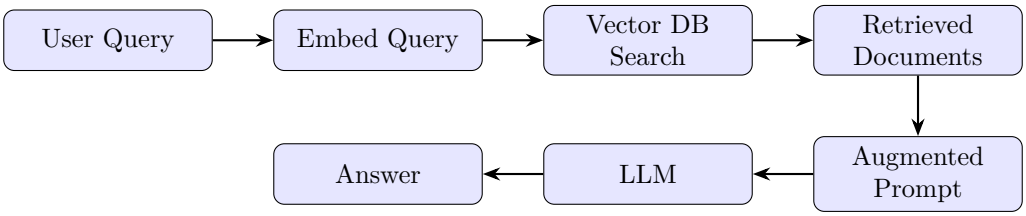
Database	Type	Key Feature
Pinecone	Managed cloud	Serverless, easy to use
Weaviate	Open-source	Hybrid search (vector + keyword)
Chroma	Open-source	Lightweight, good for prototyping
Qdrant	Open-source	Rust-based, fast
Milvus	Open-source	Scalable, enterprise-grade
pgvector	Extension	PostgreSQL extension for vectors

10 RAG (Retrieval-Augmented Generation)

10.1 The Problem RAG Solves

LLMs have a knowledge cutoff and can hallucinate. RAG grounds generation in retrieved documents.

10.2 RAG Pipeline



10.3 Key Components

1. **Document Ingestion:** Load documents → chunk them → embed chunks → store in vector DB.
2. **Chunking Strategies:**
   - Fixed-size chunks (e.g., 512 tokens) with overlap (e.g., 50 tokens).
   - Recursive character splitting (LangChain default).
   - Semantic chunking (split at topic boundaries).
3. **Retrieval:** Embed query → find top-k similar chunks.
4. **Generation:** Concatenate retrieved chunks with query → send to LLM.

10.4 Advanced RAG Techniques

- **Hybrid Search:** Combine vector similarity with BM25 keyword search.
- **Re-ranking:** Use a cross-encoder to re-rank retrieved chunks for relevance.
- **Query Transformation:** Rephrase or decompose complex queries.
- **HyDE (Hypothetical Document Embeddings):** Generate a hypothetical answer, embed it, and use that for retrieval.
- **Multi-query RAG:** Generate multiple query variants and retrieve for each.
- **Agentic RAG:** An agent decides when/what to retrieve, can do multi-step retrieval.

11 Indexing and Searching (General)

11.1 Traditional Indexing

- **Inverted Index:** Maps terms → document IDs. Core of Elasticsearch, Lucene.
- **B-Tree / B+ Tree:** Used in databases (PostgreSQL) for ordered data.
- **Hash Index:** $O(1)$ lookup; not suitable for range queries.

11.2 Text Search

- **TF-IDF:** Term Frequency × Inverse Document Frequency. Captures word importance.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

- **BM25:** Improved TF-IDF with document length normalization. Used in Elasticsearch.
- **Limitations:** Cannot capture semantic similarity ("car" vs. "automobile").

11.3 Semantic Search

- Embed documents and queries into the same vector space.
- Use ANN algorithms (HNSW, IVF) for fast retrieval.
- Models: Sentence-BERT, OpenAI text-embedding-ada-002, Cohere embed, BGE.

11.4 Hybrid Search

Combines keyword (BM25) and semantic (vector) search with score fusion:

$$\text{score}_{\text{final}} = \alpha \cdot \text{score}_{\text{BM25}} + (1 - \alpha) \cdot \text{score}_{\text{vector}}$$

12 Tool Calling

12.1 What Is Tool Calling?

LLMs can be given access to external **tools** (functions, APIs, databases). The model generates a structured function call (JSON) that the system executes, and the result is fed back.

12.2 How It Works

1. Define tools with name, description, and parameter schema (JSON Schema).
2. Include tool definitions in the system prompt or API call.
3. LLM decides when to call a tool and generates the function call with arguments.
4. System executes the function and returns the result.
5. LLM uses the result to formulate the final answer.

12.3 Example (OpenAI Function Calling)

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            },
            "required": ["city"]
        }
    }
}]
# LLM response: {"name": "get_weather", "arguments": {"city":
    "Boise"}}
```

13 A2A (Agent-to-Agent Protocol)

13.1 What Is A2A?

A2A is Google's open protocol for communication between AI agents. It defines how agents discover each other's capabilities, negotiate tasks, and exchange information.

13.2 Key Concepts

- **Agent Card:** A JSON metadata file (at `/.well-known/agent.json`) describing an agent's capabilities, skills, and supported input/output formats.
- **Task:** The unit of work. Has states: `submitted` → `working` → `completed/failed`.
- **Message/Part:** Communication units (text, file, structured data).
- **Push Notifications:** Agents can subscribe to task updates via webhooks.

13.3 A2A vs. MCP

Aspect	A2A	MCP
Purpose	Agent-to-agent communication	Agent-to-tool connection
Scope	Inter-agent collaboration	Extend single agent's capabilities
Analogy	Agents talking to each other	Agent using tools/APIs
Initiated by	Google	Anthropic

14 MCP (Model Context Protocol)

14.1 What Is MCP?

MCP (by Anthropic) is an open standard for connecting LLMs to external data sources and tools. Think of it as a **USB-C port for AI** — one protocol to connect to any tool.

14.2 Architecture

- **MCP Host:** The application (IDE, chatbot) that wants to access tools.
- **MCP Client:** Maintains a 1:1 connection with a server.
- **MCP Server:** Exposes capabilities (tools, resources, prompts) to the client.

14.3 What MCP Servers Expose

1. **Tools:** Functions the LLM can call (e.g., search database, send email).
2. **Resources:** Data the LLM can read (e.g., files, database records).
3. **Prompts:** Predefined prompt templates.

14.4 Transport Protocols

- **stdio:** For local processes (agent and server on same machine).
- **HTTP + SSE (Server-Sent Events):** For remote servers.

14.5 Why MCP Matters

- Before MCP: Each tool integration required custom code (N models $\times$ M tools $= N \times M$ integrations).
- With MCP: Each model implements MCP client once, each tool implements MCP server once ($N + M$ integrations).
- Adopted by: Cursor, VS Code Copilot, Claude Desktop, Windsurf.

15 Agentic AI Frameworks

15.1 LangChain

LangChain Core Concepts

- **LLMs / Chat Models:** Wrappers around LLM APIs (OpenAI, Anthropic, Ollama).
- **Prompt Templates:** Structured prompts with variables.
- **Chains:** Sequential pipelines (prompt $\rightarrow$ LLM $\rightarrow$ output parser).
- **Agents:** LLM-powered decision makers that choose which tools to use.

- **Tools:** Functions agents can call (search, calculator, SQL).
- **Memory:** Conversation history management.
- **Retrievers:** Interface to vector stores for RAG.
- **Output Parsers:** Structure LLM output (JSON, lists).
- **LCEL (LangChain Expression Language):** Composable chain syntax using `|` pipe operator.

15.2 LangGraph

- Built on LangChain; models agent workflows as **directed graphs**.
- **Nodes:** Functions/agents that perform work.
- **Edges:** Control flow (including conditional routing).
- **State:** Shared state object passed between nodes.
- **Persistence:** Built-in checkpointing for long-running workflows.
- **Human-in-the-loop:** Can pause execution for human approval.
- Ideal for complex, multi-step agentic workflows with cycles and branching.

15.3 CrewAI

- Multi-agent framework with **role-based** design.
- **Agents:** Each has a role, goal, and backstory.
- **Tasks:** Assigned to specific agents with expected output.
- **Crew:** Orchestrates agents to complete tasks.
- **Process types:** Sequential, hierarchical (manager agent delegates).
- Simple API; good for quick multi-agent prototyping.

15.4 AutoGen (Microsoft)

- Framework for building multi-agent **conversations**.
- Agents communicate via messages (like a group chat).
- **AssistantAgent:** LLM-powered agent.
- **UserProxyAgent:** Executes code, represents human.
- Supports code generation, execution, and debugging in conversation.
- Good for collaborative problem-solving between agents.

15.5 n8n

- **Visual workflow automation** platform (low-code/no-code).
- Drag-and-drop nodes to build workflows.
- 400+ integrations (Slack, Gmail, databases, APIs).
- AI features: LLM nodes, AI agent nodes, vector store integration.
- Self-hostable (open-source) or cloud.
- Good for: Non-developers building AI-powered automations.

15.6 Framework Comparison

Feature	LangChain	LangGraph	CrewAI	AutoGen
Paradigm	Chains/Agents	Graph-based	Role-based	Conversation
Multi-agent	Basic	Advanced	Native	Native
State mgmt	Manual	Built-in	Automatic	Message-based
Complexity	Medium	High	Low	Medium
Best for	RAG, tools	Complex flows	Team tasks	Code tasks

16 Orchestration

16.1 What Is Agent Orchestration?

The coordination layer that manages how agents execute tasks, share state, and interact.

16.2 Orchestration Patterns

1. **Sequential:** Agents execute one after another. Output of one feeds into the next.
2. **Parallel:** Multiple agents execute simultaneously on independent sub-tasks.
3. **Hierarchical:** A manager agent delegates to worker agents.
4. **Router:** A routing agent decides which specialist agent handles a query.
5. **Feedback Loop:** Agents iterate until quality criteria are met (reviewer + writer pattern).

16.3 Key Orchestration Concerns

- **Error handling:** Retries, fallbacks, graceful degradation.
- **State management:** Shared state vs. message passing.
- **Token budget:** Managing context window limits across multi-step workflows.
- **Observability:** Logging, tracing (LangSmith, Arize Phoenix).
- **Guardrails:** Ensuring agents stay within bounds (safety, cost limits).

17 Memory in AI Agents

17.1 Types of Memory

1. **Short-term / Working Memory:** The current conversation context window. Limited by model's max tokens (4K–128K+).
2. **Long-term Memory:** Persisted information across sessions.
   - **Vector store:** Embed and store past interactions; retrieve relevant ones.
   - **Key-value store:** Store structured user preferences.
   - **Graph memory:** Knowledge graph of entities and relationships.
3. **Episodic Memory:** Specific past experiences/interactions ("Last time user asked about X").
4. **Semantic Memory:** General knowledge/facts ("Paris is the capital of France").
5. **Procedural Memory:** How to do things (stored as tools/skills).

17.2 Memory in LangChain

- **ConversationBufferMemory:** Store full conversation.
- **ConversationSummaryMemory:** Summarize older messages.
- **ConversationBufferWindowMemory:** Keep only last k messages.
- **VectorStoreRetrieverMemory:** Retrieve relevant past messages via similarity search.

18 Multi-Agent Systems

18.1 Why Multi-Agent?

- Complex tasks benefit from **specialization**: each agent is an expert.
- Better **scalability**: add agents for new capabilities.
- **Modularity**: easier to develop, test, and maintain individual agents.
- **Robustness**: failure of one agent doesn't crash the system.

18.2 Common Multi-Agent Patterns

1. **Supervisor/Worker:** One supervisor delegates to specialist workers.
2. **Debate:** Multiple agents argue for/against; synthesize best answer.
3. **Pipeline:** Sequential processing (researcher → writer → reviewer).
4. **Swarm:** Agents self-organize without central control (OpenAI Swarm).
5. **Planning:** A planner agent creates a task plan; executor agents carry it out.

18.3 Design Considerations

- **Communication protocol:** Shared state vs. message passing vs. A2A.
- **Trust and safety:** Agent actions should be sandboxed; human approval for critical actions.
- **Cost control:** Each agent call costs tokens; optimize for efficiency.
- **Evaluation:** How to measure quality of multi-agent outputs.

19 Project 1: ClassQR — AI-Powered Attendance System

19.1 System Overview

ClassQR is a QR-code based smart attendance system that:

- Teachers generate QR codes for lectures; students scan to mark attendance in real time.
- Backend: Node.js + Express + PostgreSQL.
- AI: Natural language queries on attendance data using LangChain.
- Analytics: Power BI dashboards.

19.2 Database Schema

Listing 1: PostgreSQL Schema for ClassQR

```sql
-- Students table
CREATE TABLE students (
  student_id   SERIAL PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  email        VARCHAR(150) UNIQUE NOT NULL,
  roll_number  VARCHAR(20) UNIQUE NOT NULL,
  department   VARCHAR(50),
  created_at   TIMESTAMP DEFAULT NOW()
);

-- Teachers table
CREATE TABLE teachers (
  teacher_id  SERIAL PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  department  VARCHAR(50)
);

-- Courses table
CREATE TABLE courses (
  course_id   SERIAL PRIMARY KEY,
  course_name VARCHAR(100) NOT NULL,
  course_code VARCHAR(20) UNIQUE NOT NULL,
  teacher_id  INTEGER REFERENCES teachers(teacher_id)
);

-- Enrollments (many-to-many: students <-> courses)
CREATE TABLE enrollments (
  enrollment_id SERIAL PRIMARY KEY,
  student_id    INTEGER REFERENCES students(student_id),
  course_id     INTEGER REFERENCES courses(course_id),
  UNIQUE(student_id, course_id)
);

-- Lectures table
CREATE TABLE lectures (
  lecture_id   SERIAL PRIMARY KEY,
  course_id    INTEGER REFERENCES courses(course_id),
  lecture_date DATE NOT NULL,
  qr_token     VARCHAR(255) UNIQUE NOT NULL,
  created_at   TIMESTAMP DEFAULT NOW()
);

```

```sql
-- Attendance records
CREATE TABLE attendance (
  attendance_id SERIAL PRIMARY KEY,
  lecture_id    INTEGER REFERENCES lectures(lecture_id),
  student_id    INTEGER REFERENCES students(student_id),
  scanned_at    TIMESTAMP DEFAULT NOW(),
  UNIQUE(lecture_id, student_id)
);
```

19.3 LangChain + PostgreSQL Integration (Node.js)

This code demonstrates the core flow: User question (chatbot) → LLM generates SQL → Query PostgreSQL → LLM interprets results → Natural language answer.

Listing 2: ClassQR — LangChain SQL Agent (Node.js)

```javascript
// classqr-ai.js
import { ChatOpenAI } from "@langchain/openai";
import { PromptTemplate } from "@langchain/core/prompts";
import { RunnableSequence } from "@langchain/core/runnables";
import { StringOutputParser } from "@langchain/core/output_parsers";
import pg from "pg";

// --- Database connection ---
const pool = new pg.Pool({
  host: "localhost",
  port: 5432,
  database: "classqr",
  user: "postgres",
  password: "password",
});

// --- LLM setup ---
const llm = new ChatOpenAI({
  modelName: "gpt-4o-mini",
  temperature: 0,
});

// --- Schema description for the LLM ---
const DB_SCHEMA = `
Tables:
- students(student_id, name, email, roll_number, department)
- teachers(teacher_id, name, email, department)
- courses(course_id, course_name, course_code, teacher_id)
- enrollments(enrollment_id, student_id, course_id)
- lectures(lecture_id, course_id, lecture_date, qr_token)
- attendance(attendance_id, lecture_id, student_id, scanned_at)
`;

// --- Step 1: Generate SQL from natural language ---
const sqlPrompt = PromptTemplate.fromTemplate(`
You are a PostgreSQL expert. Given the database schema and a
user question, generate a valid SQL query. Return ONLY the SQL
query, nothing else.

Schema:
{schema}

```

```
Question: {question}

SQL Query:
`);

// --- Step 2: Interpret results in natural language ---
const answerPrompt = PromptTemplate.fromTemplate(`
Given the user's question and the SQL query results, provide
a clear, helpful natural language answer.

Question: {question}
SQL Query: {sql}
Results: {results}

Answer:
`);

// --- Main query function ---
async function askClassQR(question) {
  // Step 1: Generate SQL
  const sqlChain = RunnableSequence.from([
    sqlPrompt,
    llm,
    new StringOutputParser(),
  ]);

  const sqlQuery = await sqlChain.invoke({
    schema: DB_SCHEMA,
    question: question,
  });

  console.log("Generated SQL:", sqlQuery);

  // Step 2: Execute SQL against PostgreSQL
  const result = await pool.query(sqlQuery);
  const rows = JSON.stringify(result.rows, null, 2);

  console.log("Query Results:", rows);

  // Step 3: Interpret results with LLM
  const answerChain = RunnableSequence.from([
    answerPrompt,
    llm,
    new StringOutputParser(),
  ]);

  const answer = await answerChain.invoke({
    question: question,
    sql: sqlQuery,
    results: rows,
  });

  return answer;
}

// --- Example usage ---
const answer = await askClassQR(
  "What is the attendance percentage of student Khushal in Data
```

```
        Structures?"
);
console.log("Answer:", answer);

// Another example
const answer2 = await askClassQR(
  "Which students have less than 75% attendance in all courses?"
);
console.log("Answer:", answer2);
```

Flow Explanation

1. User types a natural language question in the chatbot.
2. The question + DB schema is sent to the LLM via `sqlPrompt`.
3. LLM returns a SQL query string.
4. The SQL query is executed against PostgreSQL using `pg.Pool`.
5. The raw results + original question + SQL are sent to the LLM via `answerPrompt`.
6. LLM returns a human-readable natural language answer.

20 Project 2: Motive Video Search — Hybrid Semantic Search

20.1 Problem Statement

Given 500 dashcam videos with text descriptions, build a system so that a natural language query like "A white truck hitting a deer at night on a snowy road" returns matching videos.

20.2 Approaches Tried

1. **TF-IDF + Cosine Similarity:** Basic text matching. Failed because "animal" $\neq$ "deer".
2. **Vector Embeddings (Amazon Titan + Pinecone):** Semantic search captured meaning but failed on structured constraints (dates, locations, numbers).
3. **Hybrid Approach (Winner):** Use LLM to generate pandas filters for structured fields, then apply semantic similarity on the filtered subset.

20.3 Hybrid Search Code (Python)

Listing 3: Motive Video Search — Hybrid Approach

```python
# motive_video_search.py

import pandas as pd
import numpy as np
from openai import OpenAI
from pinecone import Pinecone

# --- Setup ---
client = OpenAI(api_key="your-api-key")
pc = Pinecone(api_key="your-pinecone-key")
index = pc.Index("video-index")

# --- Load video metadata ---
df = pd.read_csv("video_metadata.csv")
# Columns: video_id, description, location, time_of_day,
#          weather, vehicle_type, event_type, date

# --- Step 1: Embed all video descriptions into Pinecone ---
def get_embedding(text):
    """Get embedding vector for a text using OpenAI."""
    response = client.embeddings.create(
        input=text,
        model="text-embedding-ada-002"
    )
    return response.data[0].embedding

def index_videos(dataframe):
    """Index all video descriptions into Pinecone."""
    vectors = []
    for _, row in dataframe.iterrows():
        embedding = get_embedding(row["description"])
        vectors.append({
            "id": str(row["video_id"]),
            "values": embedding,
            "metadata": {
                "description": row["description"],
                "location": row.get("location", ""),
                "time_of_day": row.get("time_of_day", ""),
```

```
                "weather": row.get("weather", ""),
            }
        })
    # Upsert in batches
    for i in range(0, len(vectors), 100):
        index.upsert(vectors=vectors[i:i+100])
    print(f"Indexed {len(vectors)} videos.")

# --- Step 2: LLM generates pandas filter for structured fields ---
def get_structured_filter(query, schema):
    """Use LLM to generate a pandas filter expression."""
    prompt = f"""Given a video search query and a DataFrame schema,
generate a pandas filter expression to narrow down videos.
Return ONLY the Python filter expression (no explanation).
If no structured filter is needed, return "None".

DataFrame columns and sample values:
{schema}

Query: {query}

Pandas filter expression:"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response.choices[0].message.content.strip()

# --- Step 3: Semantic search on filtered subset ---
def search_videos(query, top_k=10):
    """Hybrid search: structured filter + semantic similarity."""

    schema = """
    - location: string (e.g., 'Texas', 'California')
    - time_of_day: string ('day', 'night', 'dawn', 'dusk')
    - weather: string ('clear', 'rainy', 'snowy', 'foggy')
    - vehicle_type: string ('car', 'truck', 'motorcycle')
    - event_type: string ('accident', 'near_miss', 'normal')
    - date: string (YYYY-MM-DD format)
    """

    # Step A: Get structured filter from LLM
    filter_expr = get_structured_filter(query, schema)
    print(f"LLM Filter: {filter_expr}")

    # Step B: Apply filter to narrow down candidates
    if filter_expr and filter_expr != "None":
        try:
            filtered_df = df.query(filter_expr)
        except Exception:
            filtered_df = df  # fallback to full dataset
    else:
        filtered_df = df

    print(f"Filtered to {len(filtered_df)} videos "
          f"(from {len(df)} total)")
```

```python

    # Step C: Semantic search on filtered subset
    query_embedding = get_embedding(query)

    # Get embeddings for filtered video descriptions
    similarities = []
    for _, row in filtered_df.iterrows():
        desc_embedding = get_embedding(row["description"])
        # Cosine similarity
        cos_sim = np.dot(query_embedding, desc_embedding) / (
            np.linalg.norm(query_embedding)
            * np.linalg.norm(desc_embedding)
        )
        similarities.append({
            "video_id": row["video_id"],
            "description": row["description"],
            "similarity": cos_sim
        })

    # Sort by similarity
    results = sorted(
        similarities, key=lambda x: x["similarity"], reverse=True
    )
    return results[:top_k]

# --- Example usage ---
if __name__ == "__main__":
    results = search_videos(
        "A white truck hitting a deer at night on a snowy road"
    )
    print("\nTop matching videos:")
    for i, r in enumerate(results, 1):
        print(f"  {i}. [{r['video_id']}] "
              f"(sim={r['similarity']:.3f}) "
              f"{r['description'][:80]}...")
```

How the Hybrid Approach Works

1. **Structured Filter (LLM):** The LLM examines the query and generates a pandas filter. For "truck accidents in Texas at night", it might return: `(vehicle_type == 'truck') & (location == 'Texas') & (time_of_day == 'night')`.
2. **Narrowing:** This filter reduces 500 videos to maybe 20 candidates.
3. **Semantic Search:** On the 20 candidates, compute cosine similarity between query embedding and description embeddings.
4. **Result:** Best of both worlds — structured constraints are handled deterministically, semantic meaning is captured by embeddings.

21 Potential Interview Questions and Answers

21.1 Transformer / LLM Architecture

Q: Explain the self-attention mechanism step by step.

A:

1. Each input token is projected into three vectors: Query (Q), Key (K), Value (V) using learned weight matrices.
2. Compute attention scores: dot product of Q with all K vectors, scaled by $\sqrt{d_k}$.
3. Apply softmax to get attention weights (probability distribution).
4. Multiply weights by V vectors and sum to get the output for each position.
5. In multi-head attention, this is done h times in parallel with different projections, then concatenated.

Scaling by $\sqrt{d_k}$ prevents large dot products that would push softmax into regions with vanishing gradients.

Q: What is the difference between self-attention and cross-attention?

A:

- **Self-attention:** Q, K, V all come from the same sequence. Each token attends to all tokens in the same sequence. Used in both encoder and decoder.
- **Cross-attention:** Q comes from the decoder; K and V come from the encoder output. Allows the decoder to attend to the input sequence. Used in encoder-decoder models for tasks like translation.

Q: Why do decoder-only models dominate today?

A:

- Simpler architecture (one stack instead of two).
- Unified training objective (next-token prediction).
- Scale better: more parameters can be added uniformly.
- Can handle both understanding and generation.
- Empirically: GPT-3/4, LLaMA, Mistral all outperform encoder-decoder at scale.

Q: What is KV-Cache and why is it important?

A: During autoregressive generation, the model generates one token at a time. Without KV-Cache, it would need to recompute K and V for all previous tokens at each step ($O(n^2)$ per step). With KV-Cache, K and V from previous steps are cached and reused, reducing per-step computation to $O(n)$. This is crucial for inference speed but uses significant memory (proportional to sequence length $\times$ number of layers $\times$ hidden dimension).

21.2 RAG and Agents

Q: When would you use RAG vs. fine-tuning?

A:

- **RAG:** Data changes frequently, need source attribution, don't want to retrain, proprietary/private data, factual accuracy is critical.
- **Fine-tuning:** Need to change model behavior/style, domain-specific language un-

derstanding, latency-critical (no retrieval step), task-specific optimization.
- Often **both** are combined: fine-tune a model to be better at using retrieved context.

Q: Explain your ClassQR project architecture.

A: The system has four layers:
1. **Frontend:** Students scan QR codes; teachers generate them. Web/mobile interface.
2. **API Layer:** Node.js + Express handles QR generation, attendance marking, authentication.
3. **Database:** PostgreSQL stores students, courses, lectures, attendance with proper foreign keys and uniqueness constraints.
4. **AI Layer:** LangChain orchestrates a two-step process — (a) LLM converts natural language to SQL given the schema, (b) SQL is executed on PostgreSQL, (c) results are sent back to LLM for natural language interpretation.

Key design decision: Using LLM for SQL generation instead of a fixed query builder allows flexibility for arbitrary questions without predefined query templates.

Q: How did you handle the video search challenge at Motive?

A: We evolved through three approaches:
1. TF-IDF failed because it's word-matching, not semantic.
2. Pure vector embeddings (Amazon Titan + Pinecone) captured meaning but failed on structured filters like dates and locations.
3. Our winning hybrid approach: Use LLM to generate pandas/SQL filters for structured fields (location, time, weather), then apply semantic similarity only on the filtered subset. This combined deterministic precision with semantic understanding.

The 1st place team used SQL instead of pandas — an improvement we'd adopt for production.

21.3 Agent-Specific Questions

Q: What is the difference between MCP and A2A?

A: MCP (Model Context Protocol, by Anthropic) connects a single agent to external tools and data sources. It is like giving an agent arms and hands to interact with the world. **A2A** (Agent-to-Agent, by Google) enables multiple agents to discover and communicate with each other. It is like giving agents the ability to talk to and delegate to other agents. They are complementary: an agent uses MCP to access tools, and uses A2A to collaborate with other agents.

Q: Compare LangChain and LangGraph. When to use which?

A: LangChain is for building linear chains (prompt → LLM → tool → output) and simple agents. Good for RAG, basic tool-calling agents. **LangGraph** extends LangChain for complex workflows with cycles, conditional branching, parallel execution, and persistent state. Use it when you need:
- Multi-step reasoning with loops (agent retries, reflection).
- Multiple agents coordinating.
- Human-in-the-loop workflows.

- Long-running stateful processes.

Rule of thumb: If your workflow is a straight line, use LangChain. If it's a graph with branches and loops, use LangGraph.

Q: What is Mixture of Experts and why is it important?

A: MoE replaces the standard FFN in Transformer blocks with multiple "expert" FFNs. A router network selects the top-k experts for each token. Benefits:

- More total parameters (knowledge capacity) without proportional compute increase.
- Mixtral 8x7B has 46.7B params but only activates 12.9B per token, matching LLaMA-2 70B.
- Enables scaling to trillion-parameter models efficiently.

Challenges: Load balancing (some experts get overused), higher memory requirements (all experts in memory), training instability.

22 Quick Reference: Key Formulas and Concepts

22.1 Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

22.2 Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

22.3 Positional Encoding

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

22.4 Cosine Similarity

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

22.5 TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

22.6 Chinchilla Scaling

$$D_{\text{optimal}} \approx 20 \times N$$

22.7 MoE Gating

$$y = \sum_{i=1}^{E} g_i(x) \cdot \text{Expert}_i(x), \quad g = \text{TopK}(\text{softmax}(W_g x))$$

22.8 LoRA

$$W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times d},\ r \ll d$$

Best of luck for the interview, Khushal! You've got this!